

Multilayer Perceptrons, Part 1

Advanced Machine Learning

Sam Ogden

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

Learning objectives

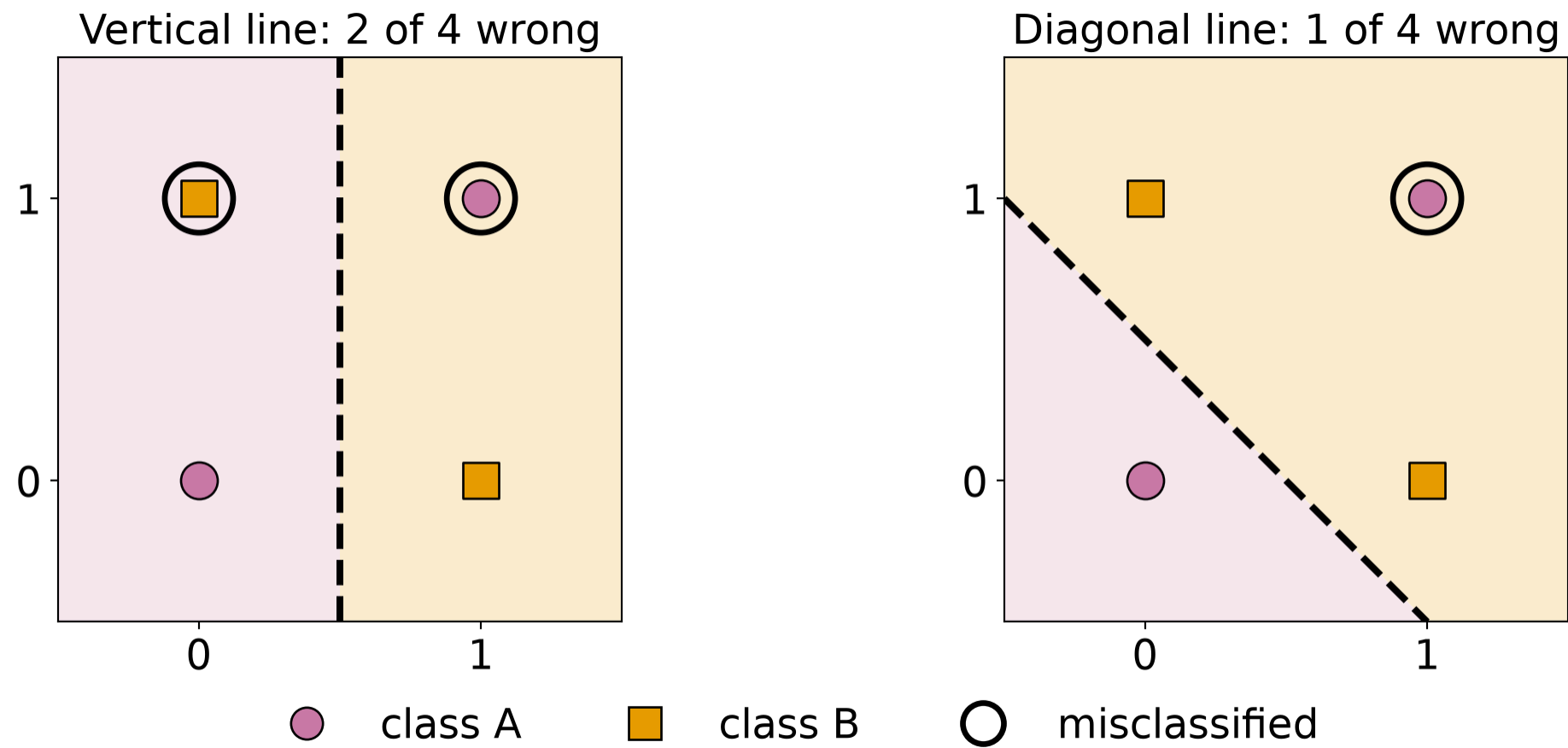
- Show why stacking linear layers alone doesn't add any power
- Introduce hidden layers and activation functions as the fix
- Assemble the full MLP forward pass, and walk one example through it by hand

The limits of a linear model

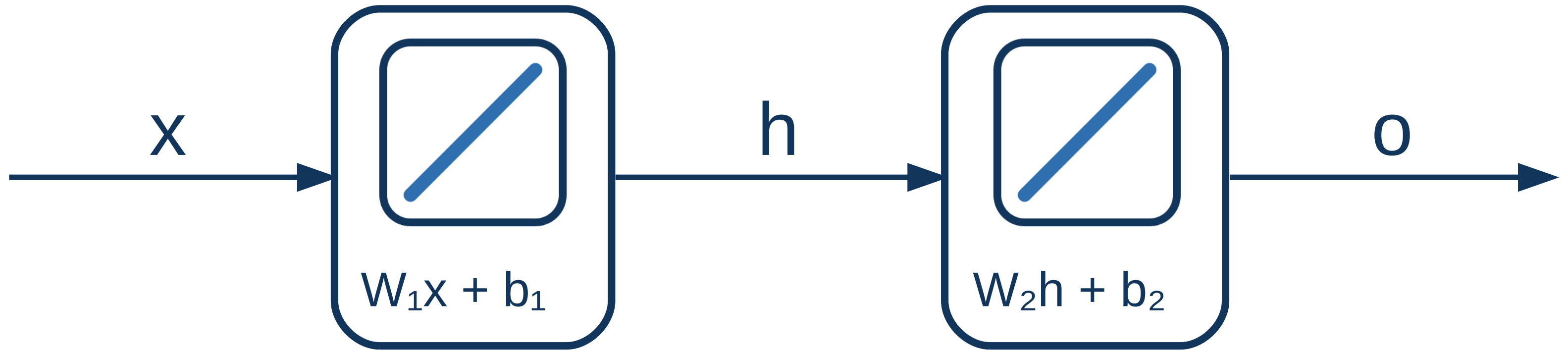
The Big Question

**Does stacking more linear layers
make a more powerful model?**

A problem no linear boundary can solve



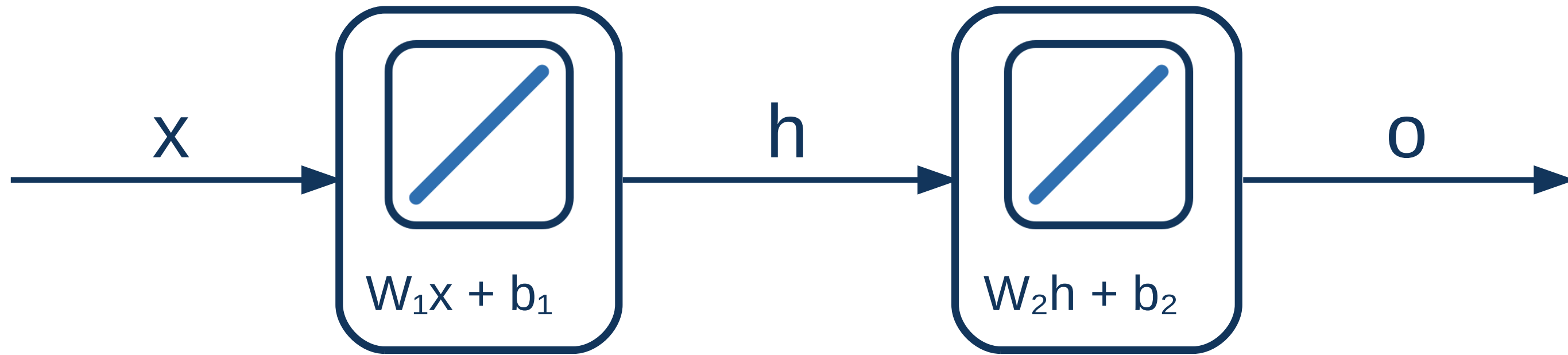
Try stacking two linear layers anyway



$$h = W_1x + b_1, \quad o = W_2h + b_2$$

- The first layer's output h becomes the second layer's input
- Does the extra layer add power? Let's follow some real numbers through it

A smaller model, with numbers



$$W_1 = \begin{bmatrix} 1 & 1 \\ -1 & 2 \end{bmatrix}, \quad W_2 = \begin{bmatrix} 1 & 1 \\ 0 & 2 \end{bmatrix}, \quad x = \begin{bmatrix} 1.2 \\ 0.2 \end{bmatrix}$$

- Same two layers as above, now with actual values
- Biases set to 0 to keep the arithmetic short — they collapse the same way, as the general case will show

Follow one input through

$$h = W_1 x = \begin{bmatrix} 1 & 1 \\ -1 & 2 \end{bmatrix} \begin{bmatrix} 1.2 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 1.4 \\ -0.8 \end{bmatrix}$$

$$o = W_2 h = \begin{bmatrix} 1 & 1 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 1.4 \\ -0.8 \end{bmatrix} = \begin{bmatrix} 0.6 \\ -1.6 \end{bmatrix}$$

- Two steps: x becomes h , then h becomes o

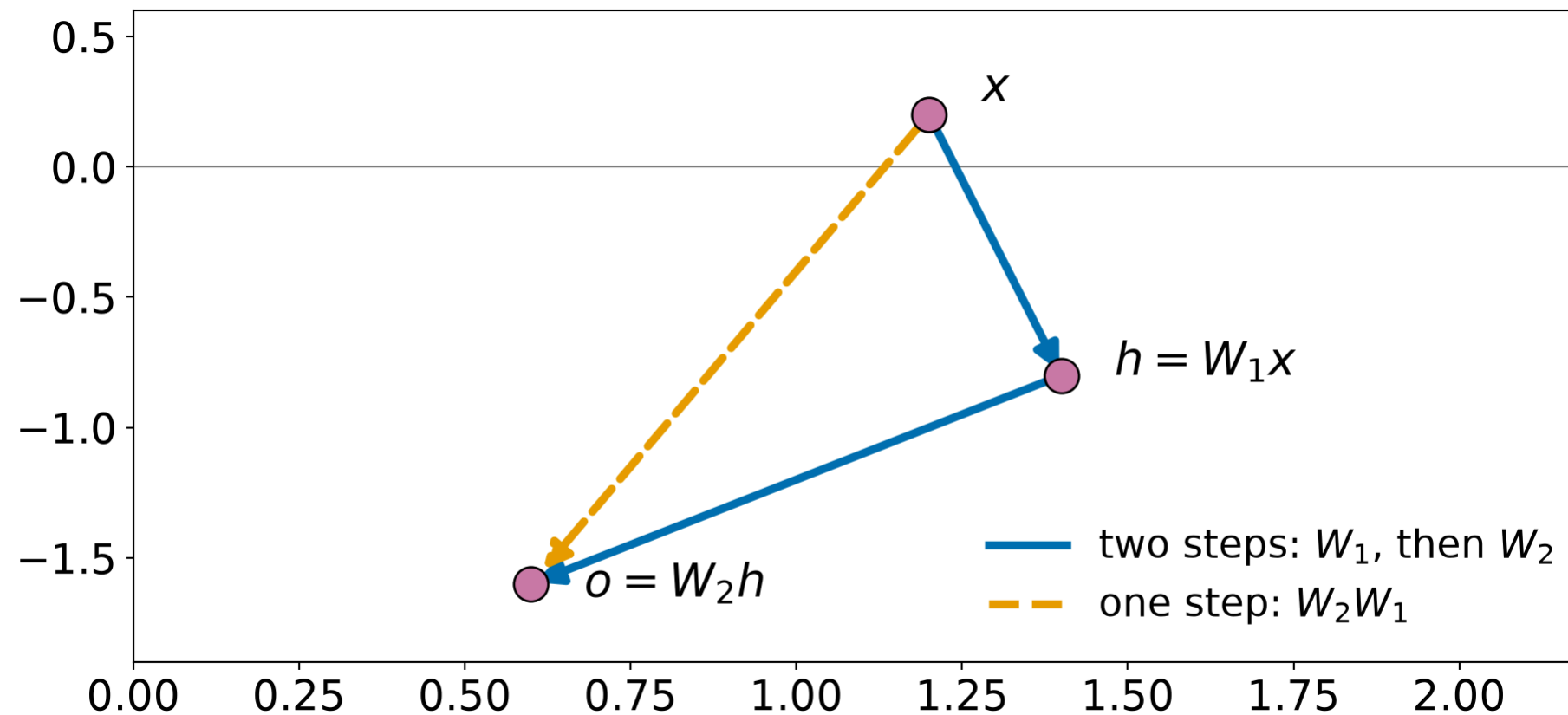
Substitute one into the other

Since $h = W_1 x$, replace h in the second equation: $o = W_2(W_1 x) = (W_2 W_1) x$

$$W_2 W_1 = \begin{bmatrix} 1 & 1 \\ 0 & 2 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ -1 & 2 \end{bmatrix} = \begin{bmatrix} 0 & 3 \\ -2 & 4 \end{bmatrix}, \quad \begin{bmatrix} 0 & 3 \\ -2 & 4 \end{bmatrix} \begin{bmatrix} 1.2 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 0.6 \\ -1.6 \end{bmatrix}$$

- One matrix, applied once, gives exactly the o that two steps did

Plotted: the same destination



- Blue: two steps through the middle. Orange dashed: one step with W_2W_1
- Both land on the same point — one linear layer did the work of two

What stacking gave us

In general, the same substitution gives:

$$o = W_2(W_1x + b_1) + b_2 = \underbrace{(W_2W_1)}_{W'} x + \underbrace{(W_2b_1 + b_2)}_{b'} = W'x + b'$$

- Two stacked linear layers are exactly one linear layer with different parameters, however wide or deep
- So whatever a single linear layer can't do — like the problem a few slides back — no stack of them can do either

Hidden layers and activation functions

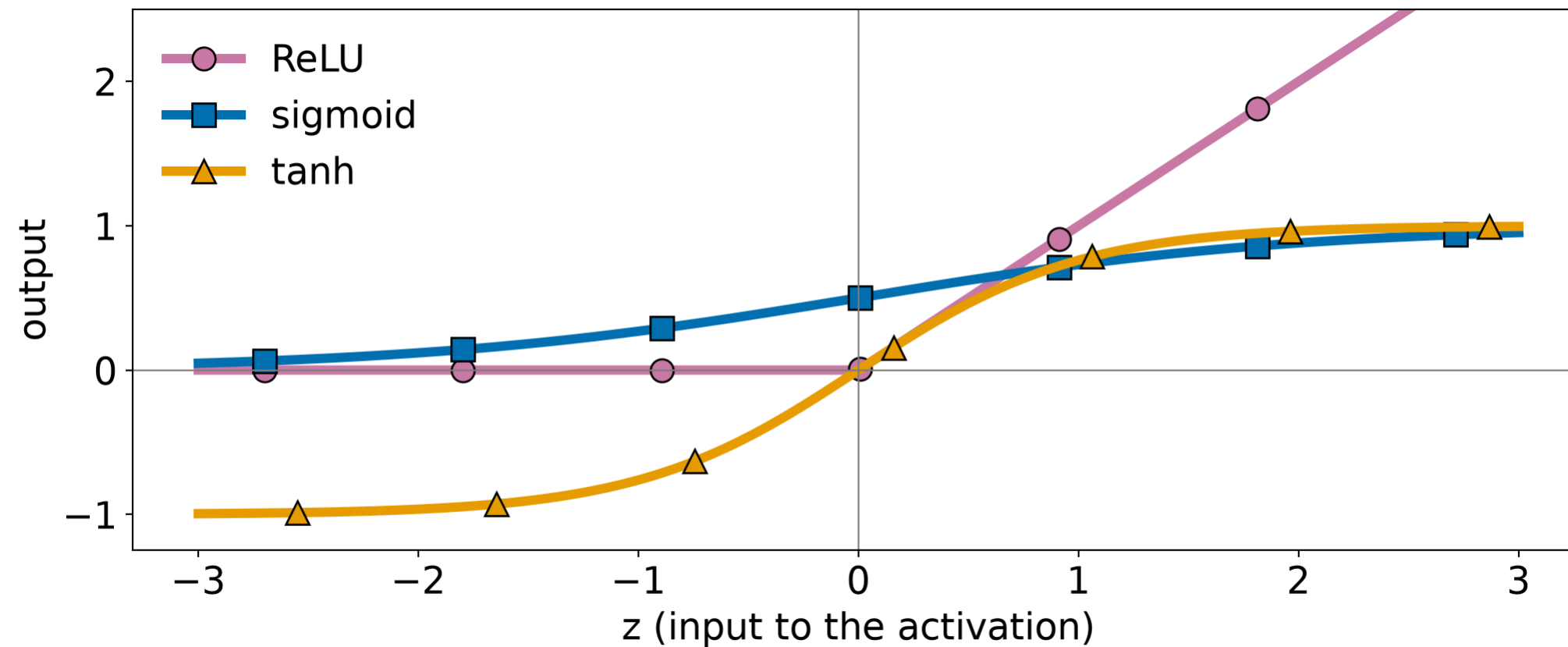
The Big Problem

**Linear-of-linear is still just linear
— we need more expressivity**

The Big Idea

Non-linear functions make linear functions independent operations

ReLU, sigmoid & tanh



- Each one **bends** — its graph isn't a straight line, which is what a linear layer can never do
- That bend is what stops stacked layers from collapsing into one: the second layer sees a reshaped h , not a straight-line function of x

Breakdown: three activation functions

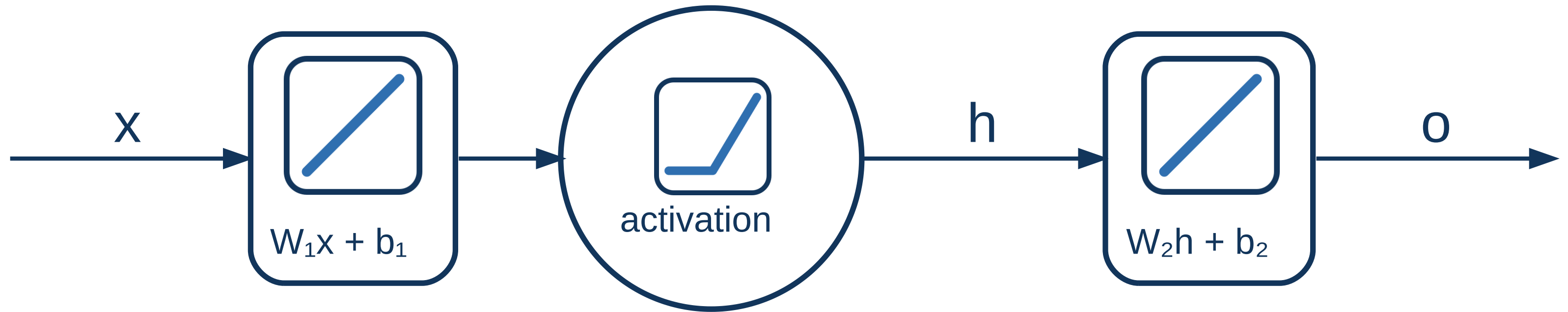
	output range	good	watch out
ReLU	$[0, \infty)$	cheap; grows for $z > 0$	zero for $z < 0$
sigmoid	$(0, 1)$	probability-like	saturates
tanh	$(-1, 1)$	centered at 0	saturates

Saturate: flatten out for large $|z|$, so the output barely changes as z changes — the slope is close to 0

Choosing one for now

- A flat curve means a tiny gradient, which means a very weak learning signal — more on why that matters in a later lecture
- $\tanh$ is just a stretched, shifted sigmoid ($\tanh z = 2 \operatorname{sigmoid}(2z) - 1$): same S-shape and same saturation, but centered at 0
- This lecture uses **ReLU** — simplest to compute by hand, and its slope (next lecture) is either 0 or 1

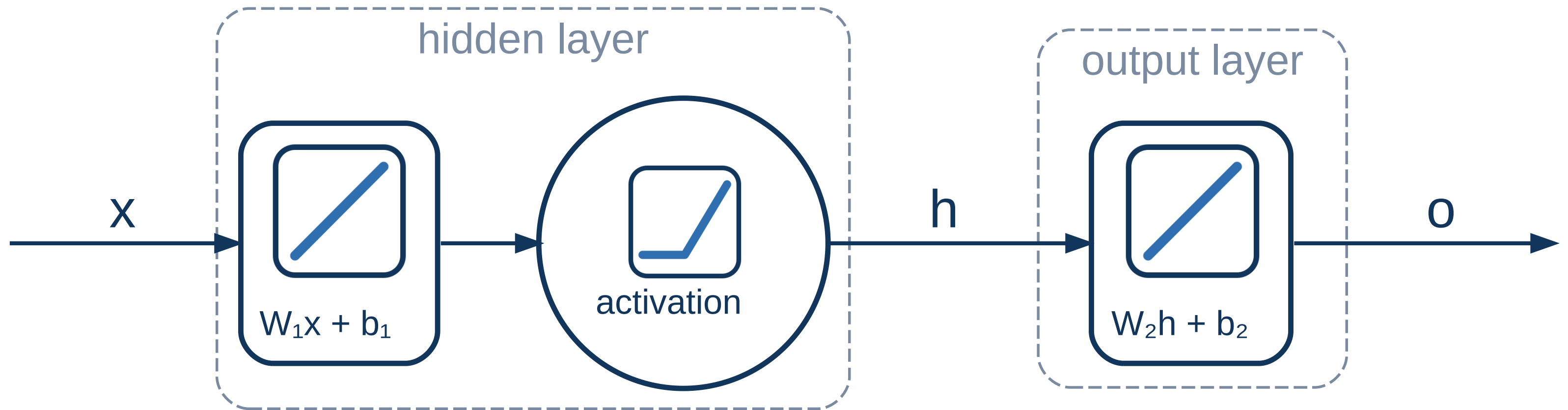
Putting an activation between the layers



$$h = \sigma(W_1x + b_1), \quad o = W_2h + b_2$$

- σ stands for *any* activation function — ReLU, sigmoid, tanh — not only the sigmoid
- It's a function applied to values, not a layer with weights of its own — hence a circle, not a box
- Layer 2 no longer sees a straight-line function of x , but a reshaped version

The hidden layer



- A **hidden layer** is a layer whose output feeds another layer instead of being the model's answer
- W_1 is the first layer of the model, yet hidden — the outside world sees only x going in and o coming out

Same model, more in the middle

```
1 class Model(nn.Module):
2     def forward(self, x):
3         h = activation(self.layer1(x))
4         # ... more model here -- we just need to pass values forward
5         return self.layer2(h)
6
7 loss = loss_fn(model(x), y)    # forward pass
8 loss.backward()               # backward pass
```

- The idea of the model hasn't changed: a **forward pass** turns x into a prediction and a loss, and a **backward pass** turns that loss into gradients for training
- Hidden layers add depth, not a new kind of training — the backward pass through that extra step is what's coming next lecture

The full forward pass

Assembling the whole pipeline

- Hidden layer:

$$h = \sigma(W_1 x + b_1)$$

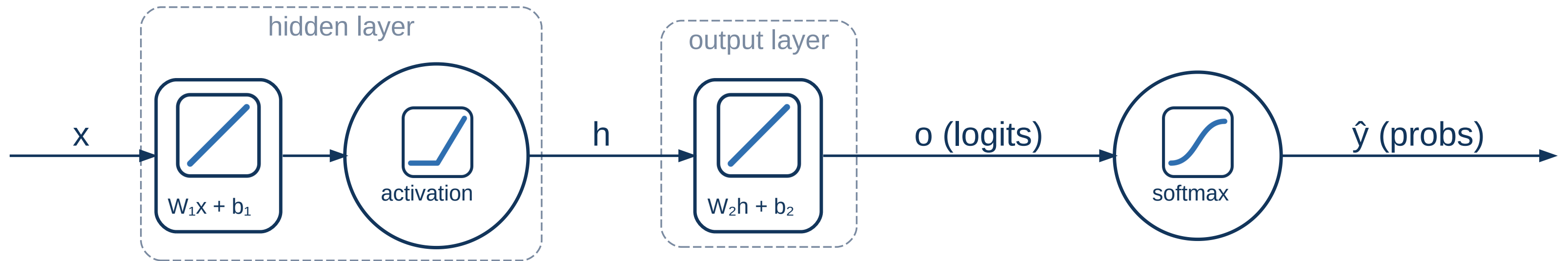
- Output layer — same shape as lecture 05's classifier, just fed h instead of x :

$$o = W_2 h + b_2$$

- Softmax, exactly as before:

$$\hat{y} = \text{softmax}(o)$$

The full picture



i Note

Hidden layer, output layer, softmax — applied one after another. This whole chain is what “forward pass” means for a network with a hidden layer

Worked example: our fruit, one layer deeper

Same task, same starting point

Same fruit classifier as lecture 05 and lab 04 — apple / banana / orange, from weight and color score — now with a 2-unit hidden layer in front of the classifier we already know. Same example row as lab 04:

$$x = [1.2, 0.2], \quad \text{true class} = \text{banana}$$

We're partway through training — here's where the parameters currently stand:

Hidden layer (W_1, b_1)

unit	row of W_1	b_1
1	[1, 1]	0
2	[-1, 2]	0

Output layer (W_2, b_2)

class	row of W_2	b_2
apple	[0.5, 1]	0
banana	[-0.3, 2]	0
orange	[0.2, -1]	0

Step 1: the hidden layer's pre-activation

Call the value before the activation h_{pre} , so that $h = \sigma(h_{\text{pre}})$:

$$h_{\text{pre}} = W_1 x + b_1$$

$$h_{\text{pre},1} = (1)(1.2) + (1)(0.2) = 1.4, \quad h_{\text{pre},2} = (-1)(1.2) + (2)(0.2) = .$$

Note

Recognize those numbers? They're exactly lab 04's logits — the same arithmetic, just relabeled. Here they're not the final answer, they're a hidden layer's raw input to ReLU

Step 2: apply ReLU

$$h = \max(0, h_{\text{pre}})$$

$$h_1 = \max(0, 1.4) = 1.4, \quad h_2 = \max(0, -0.8) = 0$$

$$h = [1.4, 0]$$

⚠ Important

Unit 2 died — its pre-activation was negative, so ReLU zeroed it. It contributes **nothing** to the output layer for this example. Keep that in mind — it matters again next lecture

Step 3: the output layer

$$o = W_2 h + b_2$$

$$o_{\text{apple}} = (0.5)(1.4) + (1)(0) = 0.7, \quad o_{\text{banana}} = (-0.3)(1.4) + (2)(0) =$$

$$o = [0.7, -0.42, 0.28]$$

- Because $h_2 = 0$, only unit 1 of the hidden layer actually influenced these logits — column 2 of W_2 contributed nothing this time

Step 4: softmax and the loss

$$\hat{y} = \text{softmax}(o) = [0.5042, 0.1645, 0.3313]$$

$$L = -\log(\hat{y}_{\text{banana}}) = -\log(0.1645) \approx 1.8048$$

- Same story as lab 04's untrained classifier: apple currently wins, even though the true label is banana
- One layer deeper, same forward-pass mechanics — every step was still just “apply this function, then the next”

Summary

What we built, and what's still missing

Model

input $\rightarrow$ hidden layer (linear + ReLU) $\rightarrow$
output layer (linear) $\rightarrow$ softmax

New idea

a hidden layer, $h = \sigma(W_1x + b_1)$, made possible by a nonlinearity

Still the same

the output layer, softmax, cross-entropy
— all exactly as in lecture 05

Warning

The open question: training needs $\nabla W_1, \nabla b_1$ — but the loss only touches W_1 *indirectly*, through h , o , and $\hat{y}$. The chain rule from lecture 05 got a gradient to one layer. Does it still work with two?

Tip

Next lecture: backpropagation — extending the chain rule through a hidden layer, worked on this exact example